

Final Exam Project Robotics

Obstacle Avoidance Robot with IoT Monitoring using Arduino Uno and ESP32

Member:

1. Kevin heryanto (001202505027)
2. Muhammad Mahasa Relung Pramardi (001202405023)
3. Susi septiyani Febrian (001202405011)
4. Rifky Nur Faishal (001202405012)
5. Hafiz Hamidy Sugianto (001202405021)
6. Alfarid Ridwan (001202405018)

1. Introduction

Robotics and Internet of Things (IoT) technologies are increasingly developing and widely implemented in automation systems. This project aims to create an obstacle avoidance robot using Arduino Uno and ESP32. The robot is capable of detecting obstacles automatically and send monitoring data through an IoT system.

The project combines robot navigation, ultrasonic sensing, servo scanning, and wireless communication through a single integrated system.

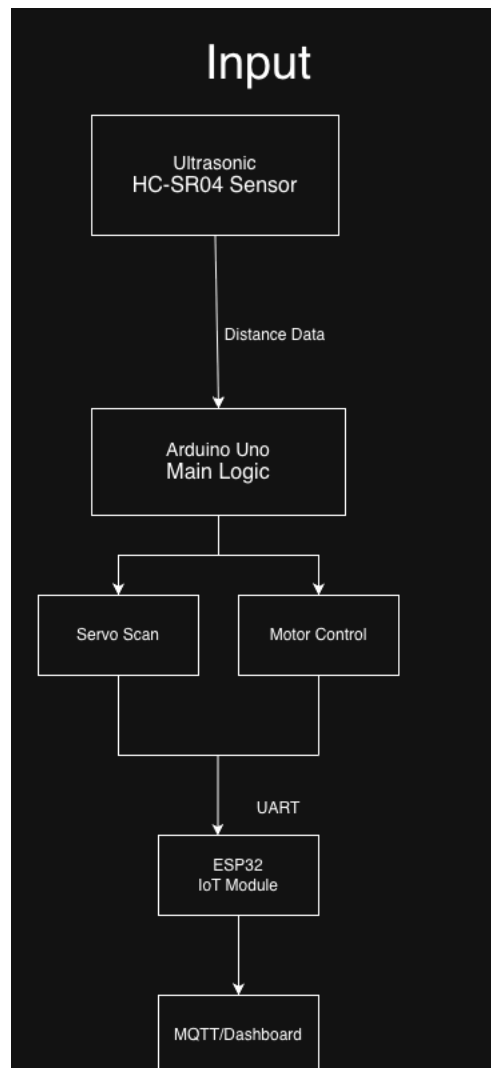
1.2 Project Objectives

The objectives of this project are:

- Build an autonomous obstacle avoidance robot using Arduino Uno. To implement obstacle avoidance using ultrasonic sensors
- Use an ultrasonic sensor and servo scanning to detect and compare obstacle distances
- Send robot status data from Arduino Uno to ESP32 using UART communication.
- Connect the ESP32 to WiFi and publish robot data through MQTT.
- Display system status on an I2C LCD connected to the ESP32.
- Prepare the system so that the simulation design can be transferred into real hardware implementation.

This project was developed and tested using the Wokwi simulation platform. The simulation environment enables collaborative development of Arduino Uno, ESP32, ultrasonic sensors, servo motors, and IoT communication without requiring physical hardware implementation.

2. System Architecture



The complete system consists of two cooperating modules. The Arduino Uno reads the ultrasonic sensor, rotates the servo to scan left and right when an obstacle is detected, controls the movement drivers, and sends formatted status messages such as `DIST=18,MODE=AUTO,TURN=LEFT` to the ESP32. The ESP32 validates the incoming message, publishes it to the MQTT topic `group2/device1/status`, and displays connection and activity information on the LCD.

The communication path between the two boards is defined over UART. In the intended hardware design, Arduino Uno TX on pin 12 connects to ESP32 RX on GPIO16, Arduino Uno RX on pin 8 connects to ESP32 TX on GPIO17, and both boards share a common ground.

2.1 Arduino Uno

- Build an autonomous obstacle avoidance robot using Arduino Uno.
- Use an ultrasonic sensor and servo scanning to detect and compare obstacle distances.
- Send robot status data from Arduino Uno to ESP32 using UART communication.
- Connect the ESP32 to WiFi and publish robot data through MQTT.
- Display system status on an I2C LCD connected to the ESP32.

The Arduino Uno source file controls the robot movement and sensing workflow. It continuously measures the front distance using the HC-SR04 sensor. When no obstacle is detected within the threshold, the robot moves forward. When an obstacle is detected within 20 cm, the robot stops, moves backward briefly, scans to the right and left using the servo-mounted ultrasonic sensor, compares the available distances, then turns toward the clearer side. After important movement changes, the Uno sends the current robot status to the ESP32 using SoftwareSerial.

2.2 ESP32

- ESP32 Dev Module as the IoT controller.
- I2C LCD for status display.
- WiFi connection using Wokwi-GUEST in simulation.
- MQTT communication using broker.hivemq.com.

The ESP32 source file handles connectivity and user-facing status output. During startup, the ESP32 initializes the serial connection, WiFi, LCD, and MQTT client. After connecting to the broker, it subscribes to grup2/device1/control and publishes status updates to grup2/device1/status.

When a UART message arrives from the Arduino Uno, the ESP32 trims the text, verifies that the message includes at least the DIST and MODE fields, then publishes the valid message to MQTT. The LCD is used to show connection status and recent message activity.

3. PlatformIO Project Structure

The repository is organized as a multi-environment PlatformIO project so the Arduino Uno and ESP32 code can exist in one workspace without interfering with each other during compilation.

Environment	Board	Framework	Main Source Used	Libraries
esp32dev	ESP32 Dev Module	Arduino	src/main_esp32.cpp	PubSubClient, LiquidCrystal_I2C
uno	Arduino Uno	Arduino	src/main_uno.cpp	Servo

The source filters in platformio.ini exclude main_uno.cpp from the ESP32 build and exclude main_esp32.cpp from the Uno build. This makes it possible to keep both programs in the same src folder while still compiling each board correctly.

4. Project Workflow

The project development process is divided into several stages:

1. Build basic robot movement
 - Forward
 - Backward
 - Left
 - Right
 - Stop
2. Implement obstacle avoidance
 - Detect obstacles
 - Stop movement
 - Move backward
 - Scan left and right
 - Select safe direction
3. Add servo scanning system
4. Develop ESP32 IoT system
 - WiFi connection
 - MQTT / HTTP testing
 - Send dummy data

5. Integrate Arduino Uno and ESP32

- UART serial communication
- Data transmission testing

5. Team Task Distribution

No	Name	Responsibility
1.	Faishal	Project Lead
2.	Kevin	Arduino Motor Control Development, Project Integration, and Bug Fixing
3.	Mahasa	Ultrasonic Sensor and Servo Scanning
4.	Ridwan	ESP32 IoT Communication
5.	Susi	UART Communication and Testing
6.	Hafiz	Documentation, Reporting, and Final Submission

6. Project Timeline

Days 1–2

- Faishal sets the system architecture, project objectives, and development deadlines.
- Kevin develops the basic robot movement functions:
 - Forward
 - Backward
 - Left
 - Right
 - Stop
- Mahas performs ultrasonic sensor and servo scanning tests.
- Ridwan validates ESP32 WiFi connectivity and initial IoT setup.
- Susi designs the UART serial communication message format.
- Hafiz prepares the report template and initial wiring diagram draft.

Days 3–4

- Kevin and Mahas integrate the obstacle avoidance logic into the Arduino Uno system.
- Ridwan develops MQTT or HTTP communication output for IoT monitoring.
- Susi tests serial communication between Arduino Uno and ESP32.
- Hafiz collects screenshots, development notes, and code documentation.
- Faishal reviews overall progress to ensure all tasks remain aligned with the project objectives.

Days 5–7

- Kevin improves robot movement stability and motor tuning.
- Mahas adjusts ultrasonic scan thresholds and servo scanning accuracy.
- Ridwan finalizes the IoT dashboard and communication output.
- Susi performs stress testing for serial logs and message parsing.
- Hafiz updates the report using actual development outputs and testing documentation.
- Faishal conducts review meetings and decides feature reductions if system stability becomes an issue.

Days 8–10

All team members focus on:

- System integration
- Bug fixing
- Communication stability
- Final debugging and synchronization between Arduino Uno and ESP32

This phase emphasizes continuous progress monitoring and collaborative integration to reduce last-minute system failures.

Days 11–14

- Freeze unstable or unnecessary features to maintain project stability.
- Rehearse the final demonstration and presentation.
- Finalize the ZIP submission package and complete the final report documentation.
- Faishal approves the final integrated system.
- Hafiz prepares and organizes the final submission files and report compilation.

7. Current Progress

Component	Status
Basic Robot Movement	Completed
Ultrasonic Sensor Testing	Completed
Servo Scanning System	Completed
ESP32 IoT Communication	Completed
UART Serial Communication	Completed
Final System Integration	Completed

8. Current Challenges

Several challenges have been identified during the current development stage are due to limitations that is in place of fully using Wokwi for simulation without access to the actual physical hardware:

- **ESP32 and Arduino Uno Integration in the real world:**
Wokwi is not able to simulate the simultaneous integration of Arduino Uno for the automated robot and ESP32 for monitoring
- **Physical feedback of how the robot handles physical terrain and obstacle avoidance:**
Wokwi is an advanced electronics and microcontroller simulator but it is not able to simulate a digital three dimensional world where our robots are able to simulate real world physical performance, therefore the logic driving the motors and obstacle avoidance are simple control logic rather than using a more advanced solution such as a PID controller

9. Testing and Results

Arduino Uno results

The Wokwi simulation shows that the Arduino Uno logic performs the intended obstacle avoidance sequence. The robot moves forward during normal conditions, stops when an obstacle is inside the threshold distance, scans right and left, and turns toward the direction with more free space. This confirms that the movement logic and decision flow are working in simulation.

ESP32 results

The ESP32 simulation works in Wokwi for WiFi connection, MQTT setup, LCD output, and message handling behavior. The LCD shows boot and connection status, and the MQTT logic is initialized as intended. Within the current project workflow, this validates the ESP32 side as the IoT monitoring controller.

Integrated result statement

The project outcome can be summarized as a successful Wokwi-based development result for both main subsystems: the Arduino Uno obstacle avoidance controller and the ESP32 IoT controller. The repository structure, code flow, UART message format, and board responsibilities are aligned with a real hardware implementation where both boards operate together as one robot system.

10. Project Limitation

The main limitation is that Wokwi does not reproduce the full Arduino Uno and ESP32 integration in one unified live robot simulation for this project structure. During

development, the Uno and ESP32 are simulated separately, so each subsystem is validated in its own context. Wokwi does not provide the exact DC motor and driver setup intended for the final robot implementation in a way that fits this project configuration. Because of that, stepper motors are used as simulation stand-ins to verify movement logic and directional control flow.

A third limitation is that due to the nature of Wokwi being an electronic and microcontroller, the simulation does not simulate any real-world terrain interaction, wheel slip, chassis balance, battery behavior, or physical motor load. Therefore, the report treats the current implementation as a logically validated prototype rather than a complete replacement for physical testing.